

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)»**

**ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ И ИНФОРМАТИКИ
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ**

**Зачётная работа
по дисциплине «Алгоритмы и структуры данных»
на тему «РАБОТА С ИЕРАРХИЕЙ ОБЪЕКТОВ: НАСЛЕДОВАНИЕ И
ПОЛИМОРФИЗМ»**

Выполнили студенты группы 3312

Лебедев И.А.

Шарапов И.Д.

Принял старший преподаватель Колинко П.Г.

Санкт-Петербург
2025

Содержание

Цель работы	3
Задание	3
Добавленные (изменённые) классы	3
Новая иерархия классов.....	4
Переопределённые функции-члены	4
Функции, сделанные недоступными.....	5
Доработанный код.....	5
Результаты работы программы.....	7
Выводы	10
Список используемых источников.....	10

Цель работы

Исследование иерархии объектов в языке C++, а именно наследования и полиморфизма.

Задание

Доработать модуль *shape.cpp*, добавив в коллекцию ещё одну фигуру – крест. Для этой фигуры нужно определить подходящее место в иерархии классов и написать недостающие функции-члены. Конструктор копии и другие генерируемые компилятором функции-члены, использование которых не предполагается, рекомендуется сделать недоступными.

Разработанной фигурой нужно дополнить картинку в указанных в позициях 1, 4, 5.

Для примыкания фигур должны использоваться их габаритные точки. Необходимо написать аналоги функции *up* (поместить *p* над *q*), обеспечивающие примыкание очередной фигуры *p* с нужной стороны по отношению к уже размещённой *q*.

В результате прогона программа должна последовательно продемонстрировать:

1. исходный набор фигур, включая добавленные, которые следует расположить так, чтобы они по возможности не перекрывались;
2. результат подготовки фигур к сборке: изменение размеров, отражения, повороты; для вновь добавленных фигур должны быть продемонстрированы все заложенные для них возможности для трансформации;
3. собранное изображение физиономии со всеми заданными добавками.

Добавленные (изменённые) классы

diagonal_cross:

Этот класс является производным от базового класса *shape* и представляет собой крест, нарисованный двумя пересекающимися диагональными линиями.

Он включает в себя:

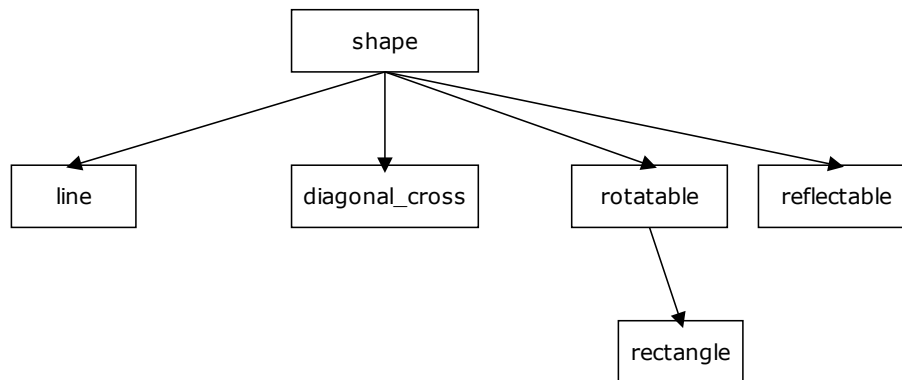
Конструктор *diagonal_cross(point, int)* — задаёт центр креста и его размер.

Метод *draw()* — рисует диагональный крест с помощью двух линий.

Метод *move(int, int)* — перемещает крест, изменяя координаты его центра.

Метод *resize(double)* — изменяет размер креста, масштабируя его пропорционально.

Новая иерархия классов



Переопределённые функции-члены

В классе *diagonal_cross* были переопределены следующие методы базового класса *shape*:

draw() — реализует отрисовку диагонального креста, состоящего из двух пересекающихся линий. Использует метод *put_line()* для соединения крайних точек.

move(int, int) — изменяет положение креста на экране, сдвигая его центральную точку *center* на заданные значения по осям *X* и *Y*.

resize(double) — изменяет размер креста, умножая его текущий размер *size* на заданный коэффициент масштабирования.

Функции, сделанные недоступными

В классе *diagonal_cross* не предусмотрены операции поворота и отражения, так как косой крест всегда имеет фиксированную ориентацию. Поэтому следующие функции из базовых классов *rotatable* и *reflectable* не реализованы и недоступны для объектов данного класса:

rotate_left() – отсутствует, так как поворот диагонального креста не изменяет его внешний вид.

rotate_right() – аналогично, не имеет смысла для данной фигуры.

flip_horizontally() – не реализован, поскольку диагональный крест остаётся неизменным при горизонтальном отражении.

flip_vertically() – также не требуется, так как вертикальное отражение не влияет на геометрию креста.

Доработанный код

```
#include <iostream>
#include "screen.h"
#include "shape.h"

class diagonal_cross : public shape {
protected:
    point center;
    int size;

public:
    diagonal_cross(point c, int s) : center(c), size(s) {
    }

    point north() const override { return point(center.x, center.y + size); }
    point south() const override { return point(center.x, center.y - size); }
    point east() const override { return point(center.x + size, center.y); }
    point west() const override { return point(center.x - size, center.y); }
    point neast() const override { return point(center.x + size, center.y - size); }
    point seast() const override { return point(center.x + size, center.y + size); }
    point nwest() const override { return point(center.x - size, center.y - size); }
    point swest() const override { return point(center.x - size, center.y + size); }

    void draw() override {
        put_line(nwest(), seast());
        put_line(neast(), swest());
    }

    void move(int a, int b) override {
        center.x += a;
        center.y += b;
    }

    void resize(double d) override {
```

```

        size *= d;
    }
};

void left(shape &p, const shape &q) {
    point w = q.west();
    point e = p.east();
    p.move(w.x - e.x - 1, w.y - e.y);
}

void right(shape &p, const shape &q) {
    point e = q.east();
    point w = p.west();
    p.move(e.x - w.x + 1, e.y - w.y);
}

void up(shape &p, const shape &q) {
    point n = q.north();
    point s = p.south();
    p.move(n.x - s.x, n.y - s.y + 1);
}

void down(shape &p, const shape &q) {
    point s = q.south();
    point n = p.north();
    p.move(s.x - n.x, s.y - n.y - 1);
}

int main() {
#ifdef LOCAL
    freopen("output.out", "w", stdout);
#endif
    setlocale(LC_ALL, "Rus");
    screen_init();

    rectangle hat(point(0, 0), point(14, 5));
    rectangle face(point(16, 0), point(28, 8));
    line brim(point(20, 10), 17);
    diagonal_cross left_ear(point(5, 15), 2);
    diagonal_cross right_ear(point(12, 15), 2);
    diagonal_cross tie(point(22, 15), 3);
    shape_refresh();
    std::cout << "=== Generated... ===\n";

    hat.rotate_right();
    brim.resize(2.0);
    face.resize(1.2);
    shape_refresh();
    std::cout << "=== Prepared... ===\n";

    face.move(-3, 10);
    up(brim, face);
    up(hat, brim);
    down(tie, face);
    left(left_ear, face);
    right(right_ear, face);
    shape_refresh();
    std::cout << "=== Ready! ===\n";
    return 0;
}

```

```

.....
.....
.....
.....*.....*.....
...*...*...*...*...*...*...
...*. *...*. *...*. *...
...*.....*.....*.....
...*. *...*. *...*. *...
...*...*...*...*...*...*...
.....*.....*.....
.....
.....*****.....
.....
.....*****.....
.....*.....*.....
.....*.....*.....
*****. *.....*.....
*. *.....*.....
*. *.....*.....
*. *.....*.....
*. *.....*.....
*****.*****.....
=== Generated... ===

```

7

```

.....
.....
.....
.....*.....*.....
...*...*...*...*...*...*...*...
....*,*....*,*....*,*....*,*....
.....*.....*.....*.....
....*,*....*,*....*,*....*,*....
...*...*...*...*...*...*...*...
.....*.....*.....
.....
...*****.....
.....*****.....
.....*.....*.....
...******,*.....*.....
....*,*....*,*....*,*....
....*,*....*,*....*,*....
....*,*....*,*....*,*....
....*,*....*,*....*,*....
....*,*....*,*....*,*....
....*,*....*,*....*,*....
...******,*****.....
=== Prepared... ===

```

Рисунок 2 – Подготовка к сборке


```

.....
.....*****.....
.....*.....*.....
.....*.....*.....
.....*.....*.....
.....*.....*.....
.....*.....*.....
.....*.....*.....
.....*****.....
...*****...
.....*****.....
.....*.....*.....
.....*.....*.....
.....*..**.....**..*.....
.....*.*. *.....*. *.....
.....*..*.....*..*.....
.....*. *.....*. *.....
.....*. *. *.....*. *. *.....
.....*..**.....**..*.....
.....*.....*.....
.....*****.....
.....*.....*.....
.....*..*.....
.....*. *.....
.....*.....*.....
.....*.....*.....
.....*.....*.....
.....
.....
.....
=== Ready! ===

```

Рисунок 3 – Сборка изображения

Выводы

В ходе выполнения работы была расширена иерархия классов, реализующих фигуры, за счёт добавления нового класса *diagonal_cross*, представляющего косой крест. Этот класс был интегрирован в существующую структуру программы, что позволило:

1. Освоить принципы наследования и полиморфизма в C++, создавая производные классы от базового *shape*.
2. Реализовать новую фигуру, которая корректно отображается и взаимодействует с другими объектами в программе.
3. Использовать переопределённые методы (*draw()*, *move()*, *resize()*) для управления свойствами креста.
4. Обеспечить корректное размещение новой фигуры, разработав вспомогательные функции для позиционирования (*left()*, *right()*, *up()*, *down()*).
5. Проверить работу программы в нескольких этапах:
 - Инициализация всех фигур, включая новый *diagonal_cross*.
 - Демонстрация изменений размеров, перемещений и трансформаций.
 - Итоговая сборка изображения с добавленной фигурой в указанных позициях.

Список используемых источников

1. Колинко П. Г. Пользовательские контейнеры: учебно-метод. пособие. СПб.: СПбГЭТУ «ЛЭТИ», 2025. 64 с. (вып.2502)